STUDENT ALLOWANCE TRACKER

ITPFE 1 — Web Systems and Technologies 2 | Prelim Project

DOCUMENTATION DRAFT — complete placeholders, actual testing, screenshots and signatures before submission.

Project title: Student Allowance Tracker (PocketPlan)

Group name: [Group name]

Members: [Full name — role; identify leader] x 3

Course and section: ITPFE 1 — Web Systems and Technologies 2, [Section]

Instructor: [Instructor name]

Date submitted: [Date]

Repository: [GitHub URL]

Live app: [GitHub Pages URL]

Video: [Video URL]

1. Project overview

Problem: Students may lose track of how much allowance they have left after daily expenses. A simple record can help them monitor spending.

Users: Students who receive an allowance and want to record purchases.

Solution: A single-page app lets students set an allowance, add or change expenses, delete entries, and view automatically calculated totals. It saves data in the same browser using localStorage.

2. Features

Feature	What the user can do	Owner
Allowance management	Enter and update allowance; see validation feedback	[Member 1]
Expense management	Add, edit, delete expenses with categories	[Member 2]
Summary	See total spent, remaining balance, spending bar	[Member 3]

3. Architecture

3.1 Folder structure: index.html; css/style.css; js/model.js; js/controller.js; js/views/View.js, AllowanceView.js, ExpenseView.js, SummaryView.js.

3.2 State object: `{ allowance: 1000, expenses: [] }`. **allowance** stores the current budget; **expenses** stores entries with id, name, amount, category, and date.

3.3 Responsibilities: Model stores and validates data, saves to localStorage, and calculates summary values. Views extend the base View and render markup from supplied data; they expose event-handler registration methods.

Controller subscribes to events, calls model functions, and re-renders the views.

4. Views

View class	Parent	Handler methods	Owner
AllowanceView	#allowance-section	addHandlerSave	[Member 1]
ExpenseView	#expense-section	addHandlerSubmit, addHandlerActions	[Member 2]
SummaryView	#summary-section	None (display only)	[Member 3]

5. Tracing one user action

1. User fills out the expense form; ExpenseView.addHandlerSubmit captures the input.

2. The handler calls controlSubmitExpense in controller.js with the form data.

3. The controller calls model.addExpense in model.js, which validates the fields, updates state.expenses, and saves to localStorage.

4. The controller calls renderAll, which invokes expenseView.render and summaryView.render.

5. The views rebuild their markup, showing the expense and updated totals.

6. Screenshots

[Insert 3–5 real screenshots of the running app. Include one showing a validation error and one showing the updated balance. Add one-line captions.]

7. Testing

Test	Input / action	Expected result	Actual / Pass?
1	Save allowance 1000	Allowance and summary update	[Fill after test]
2	Add lunch, 75, Food	Expense appears; balance decreases	[Fill after test]
3	Edit lunch to 95	Expense and total update	[Fill after test]
4	Delete expense	Expense disappears; total updates	[Fill after test]
5	Empty expense name	Validation error; no new expense	[Fill after test]
6	Expense amount -5	Validation error; no new expense	[Fill after test]
7	Refresh page	Saved allowance/expenses remain	[Fill after test]

8. Challenges and solutions

Challenge	How we solved it	What we learned
[Actual issue encountered]	[Actual fix]	[Actual lesson]

9. Individual contributions

Member	Views and functions written	Commits	Share (%)
[Name 1]	[Actual files and functions]	[Actual count]	[ ]
[Name 2]	[Actual files and functions]	[Actual count]	[ ]
[Name 3]	[Actual files and functions]	[Actual count]	[ ]

Shares must total 100%. Each member signs here: [Signatures of all members].

10. Individual reflections

[Each member writes their own 100–150 words about what they built, how MVC helped or made work harder, and what they would change.]

11. References and AI use

Course Week 5 View framework materials: [Exact instructor resource]. JavaScript documentation: [URLs actually consulted].

Member	AI tool	What you asked	Files/functions affected
[Actual member(s)]	ChatGPT	Initial project scaffold, explanations and documentation draft	index.html, CSS, model, controller, View classes, report draft

Update this disclosure to accurately describe how each member used AI. Each member must understand and explain the code they submit.